

Лекция 2. Методика диагностики Linux-сервера

Цель лекции

Понять диагностику Linux-сервера как последовательный процесс поиска причины

Научиться отличать симптом, причину и событие, которое просто произошло рядом по времени

Освоить базовый порядок проверки ресурсов, служб, портов, журналов, сети, DNS и firewall

Выполнять изменения безопасно: backup, syntax check, apply, test, rollback

Доказывать результат исправления повторной проверкой исходного пользовательского сценария

Учебные вопросы

1. Симптом, причина и сопутствующее событие
2. Baseline исправного сервера
3. CPU, RAM, Load Average, Swap и процессы
4. Диски и файловые системы
5. Службы, процессы, порты и журналы
6. Сеть, DNS и firewall
7. Backup, syntax check и rollback
8. Документирование и подтверждение исправления

1. Диагностика начинается с точной формулировки проблемы

Симптом - наблюдаемое проявление проблемы:
жалоба пользователя, ошибка мониторинга, отказ сервиса

Причина - конкретная неисправность, из-за которой появился симптом

Сопутствующее событие - действие или событие рядом по времени, которое может быть связано, а может совпасть случайно

Пример симптома: web01 не открывается по HTTP с клиентской машины

Пример причины: Nginx не запустился из-за ошибки

Диагностическая цепочка удерживает ход мысли

Симптом → Гипотеза → Команда проверки →

Фактический вывод

Фактический вывод → Причина → Исправление →

Повторная проверка

Гипотеза без команды проверки остается догадкой

Команда без вывода не доказывает причину

Исправление без повторной проверки не считается
завершенным

В отчете фиксируют не все команды подряд, а те,
которые подтвердили вывод

Событие рядом по времени не доказывает причину

Перезагрузка могла только проявить старую ошибку в конфигурации

Обновление пакета могло совпасть с заполнением диска

Рост нагрузки может быть следствием отказа службы, а не исходной причиной

После каждого события нужно проверить факты: что изменилось и что реально сломано

Корректная формулировка: событие могло повлиять, но причина подтверждается только проверкой

Симптом подсказывает первую ветку проверки

Сервер полностью недоступен → VM, link, IP, route

Сервер доступен, сервис недоступен → systemd, port, log

По IP работает, по имени нет → DNS

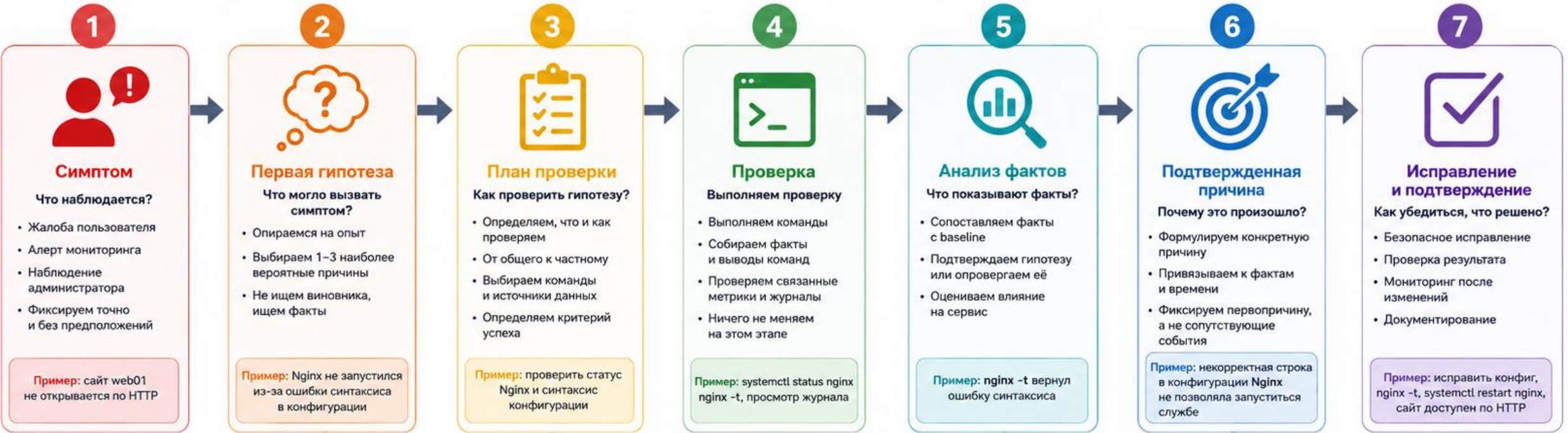
Локально работает, удаленно нет → bind, firewall, route

Служба не запускается → syntax, log, disk, permissions

Сервер медленный → CPU, RAM, I/O, application

Диаграмма: путь от симптома к подтвержденной причине

Структурированный процесс диагностики Linux-сервера от проблемы до подтверждения и исправления.



Сопутствующее событие (не равно причина)

- Событие рядом по времени может быть случайным совпадением
- Перезагрузка, обновление, рост нагрузки, заполнение диска — не всегда причина
- Проверяем гипотезу фактами, а не датой события

11:00

Перезагрузка сервера

11:45

Обновление пакетов

12:30

Рост нагрузки

13:10

Симптом: сайт не открывается

Доказательства важнее предположений

Хорошая запись	Плохая запись
Симптом: сайт не открывается по HTTP	Наверное, проблема в сети
Гипотеза: Nginx не запустился	Скорее всего, упал Nginx
Команда: <code>systemctl status nginx</code>	Проверил что-то в консоли
Вывод: <code>nginx.service failed (code=exited, status=1)</code>	Что-то с ошибкой
Вывод администратора: ошибка синтаксиса в <code>/etc/nginx/nginx.conf:42</code>	Похоже, конфиг битый

Итог

Гипотеза подтверждена

- Исправление
- Проверка результата
- Мониторинг
- Документирование

Гипотеза опровергнута

- Формулируем новую гипотезу
- Сужаем область поиска
- Возвращаемся к шагу 3
- Не расширяем список догадок

2. Baseline показывает нормальное состояние сервера

Baseline = зафиксированное нормальное состояние сервера

Он создается после рабочей настройки и обновляется после плановых изменений

Без baseline трудно понять, является ли текущее значение нормой или отклонением

Baseline помогает сравнить ресурсы, сеть, маршруты, службы и открытые порты

Для учебной работы достаточно текстового файла с основными командами

Минимальный baseline для junior-администратора

date, hostnamectl, uptime - время проверки, имя узла и время работы

free -h - оперативная память RAM и swap

df -hT, df -i, lsblk -f - файловые системы, место, inode и блочные устройства

ip -br addr, ip route - адреса интерфейсов и маршруты

ss -ltnur - слушающие TCP/UDP-порты и процессы

systemctl --failed - службы, завершившиеся с ошибкой

Baseline нужен для сравнения, а не для красивого архива

Текущие значения сравнивают с тем, что было на исправном сервере

Если раньше TCP 80 слушался на 0.0.0.0, а теперь только на 127.0.0.1, это важное отличие

Если раньше default route был через 192.168.10.1, а теперь отсутствует, это возможная причина недоступности

Если раньше `systemctl --failed` был пустым, а теперь есть failed-служба, ее проверяют первой

Не нужно строить сложный one-liner: важнее понимать, что именно сохраняется и зачем

Схема: baseline как точка сравнения при диагностике

Baseline — это зафиксированное нормальное состояние сервера, с которым сравнивают текущее состояние при поиске причины проблемы.



Почему baseline полезен



1. Показывает норму именно этого сервера



2. Позволяет отличить симптом от отклонения



3. Ускоряет поиск причины



4. Делает диагностику проверяемой



Пример

Сегодня сайт отвечает медленно. Baseline показывает, что обычно load 0.5–1.0, а сейчас load 6.8 и высокий %wa — значит, внимание нужно сместить на диск или storage, а не только на CPU.

3. Load Average не равен загрузке CPU

CPU (Central Processing Unit) - центральный процессор, выполняющий инструкции ОС и программ

Load Average - среднее количество задач, готовых выполняться или ожидающих недоступный ресурс

В Linux высокий load может быть связан не только с CPU, но и с ожиданием диска или другого I/O

Команда uptime показывает load average за 1, 5 и 15 минут

Load оценивают относительно числа логических CPU и baseline данного сервера

CPU проверяют по нескольким признакам

uptime - быстро показывает load average

top - показывает процессы и распределение CPU по режимам

vmstat 1 - показывает динамику CPU, очередей и ожидания I/O каждую секунду

%us - время CPU на пользовательские приложения

%sy - время CPU на работу ядра

%id - простой CPU, %wa - ожидание I/O, %st - steal time виртуальной машины

Высокий load требует уточнения причины

Если %id близок к 0 и %us высок, вероятно нагрузка приложений на CPU

Если %wa высокий, задачи могут ждать диск, а не процессор

Если в vmstat растёт очередь r, процессы ждут CPU

Если в vmstat растёт b, процессы могут ждать I/O

Вывод администратора должен звучать конкретно:

CPU-загрузка, I/O-ожидание или смешанная нагрузка

Процесс сначала идентифицируют, а не сразу завершают

`ps aux --sort=-%cpu | head` - показывает процессы с максимальной CPU-нагрузкой

`ps aux --sort=-%mem | head` - показывает процессы с максимальным потреблением памяти

PID (Process ID) - числовой идентификатор процесса в системе

Перед остановкой нужно понять назначение процесса: служба, пользовательская задача, системный процесс

Для службы `systemd` корректнее использовать `systemctl`, а не `kill` по PID

Остановку выполняют самым безопасным способом

`sudo systemctl restart nginx` - перезапускает службу

Nginx через `systemd`

`kill -TERM <PID>` - просит обычный процесс корректно завершиться

`kill -KILL <PID>` - принудительно завершает процесс без возможности штатной очистки ресурсов

`SIGKILL` используют только если нормальное завершение не работает

После остановки или `restart` всегда проверяют, исчез ли симптом и не появился ли новый отказ

Свар используется не только при аварии

RAM (Random Access Memory) - оперативная память, с которой работает процессор

Свар - область диска, куда Linux может временно переносить страницы памяти

free -h показывает available RAM, used, buff/cache и swap

vmstat 1 показывает si и so: обмен страницами между RAM и swap

Проблема подтверждается сочетанием: мало available RAM, активные si/so, задержки и события OOM

Сам факт использования swap не означает аварию

OOM Killer фиксирует критическую нехватку памяти

OOM (Out Of Memory) - ситуация, когда системе не хватает памяти для продолжения работы процессов

OOM Killer - механизм ядра Linux, который завершает один из процессов, чтобы освободить память

`journalctl -k -b | grep -i -E 'oom|out of memory|killed process'` - ищет OOM-события текущей загрузки

Если процесс завершен OOM Killer, простого restart может быть недостаточно

Нужно найти, почему память росла: приложение, нагрузка, утечка, слишком маленькая VM

4. Диски проверяют кратко и по делу

`df -h` - показывает свободное и занятое место на файловых системах

`df -i` - показывает использование inode

`lsblk -f` - показывает блочные устройства, типы файловых систем и точки монтирования

`findmnt` - показывает, откуда и с какими параметрами смонтированы файловые системы

`journalctl -k` - помогает увидеть ошибки ядра, связанные с диском или файловой системой

В этой лекции диск рассматривается как часть диагностики. без повторения всей темы хранения

Поиск заполненного каталога начинают с границы файловой системы

`sudo du -xhd1 /var` - показывает размер каталогов
первого уровня внутри `/var`

Ключ `-x` не дает перейти на другую файловую систему

Если закончились `inode`, проблема может быть не в
больших файлах, а в множестве мелких файлов

Перед удалением проверяют назначение файла и
процесс, который его использует

Удаление логов без понимания причины может скрыть
следы проблемы

Дополнительно: df и du могут расходиться

Ситуация: df показывает занятое место, а du не находит такой объем в каталогах

Частая причина - файл удален из каталога, но процесс продолжает держать его открытым

sudo lsof +L1 - показывает открытые удаленные файлы

Это production-кейс, его достаточно знать как дополнительную ветку диагностики

Место освобождается после закрытия файла процессом или перезапуска соответствующей службы

Read-Only filesystem ломает запись даже при свободном месте

Read-Only - режим, в котором файловая система доступна только для чтения

`findmnt /` - показывает параметры монтирования корневой файловой системы

Если в `options` есть `ro`, запись в эту файловую систему невозможна

`journalctl -k -b -p warning` - помогает найти предупреждения ядра текущей загрузки

Главная мысль: свободное место не гарантирует возможность записи

5. active в systemd не доказывает работу приложения

systemd может считать службу запущенной, но приложение может отвечать ошибкой

Служба может быть active, но слушать только localhost

Порт может слушаться, но приложение может возвращать HTTP 500

Поэтому проверка идет по модели Service → Port → Application

Каждый уровень подтверждает только свой участок цепочки

Service → Port → Application проверяются разными командами

`systemctl is-active nginx` - проверяет состояние службы

Nginx в systemd

`sudo ss -ltnp | grep :80` - проверяет, слушается ли TCP-порт 80 и каким процессом

`curl http://127.0.0.1/` - проверяет HTTP-ответ

приложения локально

Если `service failed`, сначала читают журнал и `syntax check`

Если `service active`, но port не слушается, проверяют конфигурацию `bind/listen`

Bind Address объясняет разницу между localhost и сетью

127.0.0.1:80 - сервис доступен только локально на сервере

0.0.0.0:80 - сервис слушает все IPv4-интерфейсы сервера

192.168.10.20:80 - сервис слушает конкретный IP-адрес

Если curl на localhost работает, а клиент не подключается, проверяют bind, firewall и route

Bind Address - адрес, на котором процесс принимает входящие соединения

Журналы читают вокруг времени симптома

`journalctl -u nginx -b` - показывает журнал службы Nginx за текущую загрузку

`journalctl -u nginx --since '-30 min'` - ограничивает журнал последними 30 минутами

Журнал ищут не с конца случайно, а вокруг времени, когда появился симптом

В отчет попадают только строки, подтверждающие причину

Дополнительно: `journalctl -u nginx -b -1` полезен только если журнал предыдущей загрузки сохраняется

Syntax check выполняется до применения конфигурации

`sudo nginx -t` - проверяет конфигурацию Nginx без перезапуска службы

`sudo sshd -t` - проверяет синтаксис конфигурации OpenSSH Server

Модель безопасного изменения: Edit → Syntax Check → Reload → Test

`reload` перечитывает конфигурацию без полной остановки процесса

`restart` полностью останавливает и запускает службу заново

Если служба остановлена, `reload` обычно не заменяет

Изменение SSH требует отдельной страховки

SSH (Secure Shell) - защищенный удаленный доступ к серверу по командной строке

Перед применением `sshd_config` выполняют `sudo sshd -t`

Текущую SSH-сессию не закрывают до проверки новой конфигурации

Открывают вторую SSH-сессию и проверяют вход

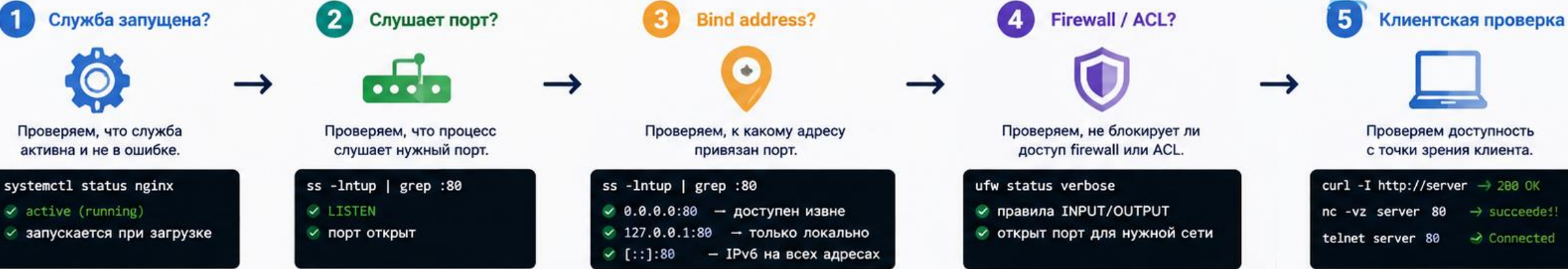
В лаборатории полезна консоль VirtualBox или

PNetLab, если удаленный вход сломан

Ошибка в SSH может заблокировать администратора вне сервера

Схема: служба, порт, bind address и клиентская проверка

Путь проверки: служба запущена → слушает порт → привязана к адресу → доступна клиенту



Как интерпретировать bind address			
Bind address	Пример в ss	Значение	Клиенты
0.0.0.0	0.0.0.0:80	Слушает на всех IPv4-адресах сервера	Доступен локально и извне (IPv4)
127.0.0.1	127.0.0.1:80	Слушает только на локальном интерфейсе (loopback)	Доступен только локально
[::]	:::80	Слушает на всех IPv6-адресах	Доступен локально и извне (IPv6)
192.168.1.10	192.168.1.10:80	Слушает только на конкретном интерфейсе	Доступен только из сети этого интерфейса

Пример: Nginx на порту 80		
	1. Служба	<pre>systemctl status nginx</pre> active (running) ✓
	2. Порт	<pre>ss -ltnup grep :80</pre> LISTEN 0 128 0.0.0.0:80 users:("nginx",pid=1234,fd=6) ✓
	3. Bind	Привязан к 0.0.0.0:80 → доступен извне (IPv4)
	4. Firewall	<pre>ufw status verbose</pre> 80/tcp ALLOW IN Anywhere ✓
	5. Клиент	<pre>curl -I http://server</pre> HTTP/1.1 200 OK ✓

Если не доступно — куда смотреть		
	Служба не запущена	→ <pre>systemctl status <service></pre> <pre>journalctl -u <service> -xe</pre>
	Порт не слушается	→ конфигурация службы ошибка запуска
	Bind только на 127.0.0.1 или конкретном IP	→ изменить конфиг (listen/bind) перезапустить службу
	Firewall блокирует	→ открыть порт / настроить правила
	Клиент не может подключиться	→ маршрут, DNS, ACL, Cloud Security Group и т.д.

Полезные команды

Службы

```
systemctl status <service>  
systemctl is-enabled <service>  
journalctl -u <service> -n 50 --no-pager
```

Порты и процессы

```
ss -ltnup  
ss -ltnup | grep :80  
lsof -iTCP -sTCP:LISTEN -P -n
```

Сеть

```
ip -br a  
ip route  
ip addr show
```

Firewall (UFW / iptables)

```
ufw status verbose  
iptables -S  
iptables -L -n -v
```

Клиентские проверки

```
curl -I http://server  
nc -vz server 80  
telnet server 80
```

Итог

Услуга доступна клиенту, когда:
служба запущена, порт слушается,
привязка позволяет доступ, firewall
не блокирует и клиент получает ответ.

6. Сетевую диагностику начинают с локального интерфейса

Link - физическая или виртуальная связность интерфейса

ip -br link - показывает состояние интерфейсов
UP/DOWN

ip -br addr - показывает IP-адреса и маски интерфейсов

ip route - показывает таблицу маршрутизации

ip neigh - показывает соседние узлы в ARP/Neighbor-таблице

Порядок проверки: Link → Address → Gateway → Route
→ Remote Host

Маршрут проверяют на управляемой цели

Сначала проверяют адрес сервера и состояние интерфейса

Затем проверяют gateway своей сети, например 192.168.10.1

После этого проверяют удаленный сервер в другой подсети лаборатории

`ip route get 192.168.30.10` - дополнительно показывает выбранный маршрут и исходный IP

`ip route get` не доказывает доставку пакета до конца, а показывает решение локальной таблицы

маршрутизации

DNS проверяют отдельно от IP-доступности

DNS (Domain Name System) - система преобразования доменных имен в IP-адреса

`getent hosts web01.company.test` - проверяет системное разрешение имени

`dig @192.168.30.53 web01.company.test` - отправляет запрос конкретному DNS-серверу

`cat /etc/resolv.conf` - показывает, какие DNS-серверы использует система

`getent ahosts` и `resolvectl` можно использовать дополнительно, если они есть в системе

Логика простая: по IP работает, по имени нет →

Firewall проверяют через механизм управления

Firewall - механизм фильтрации входящего, исходящего или транзитного трафика

Ubuntu: `sudo ufw status verbose` - показывает состояние UFW и правила

RHEL-like: `sudo firewall-cmd --list-all` - показывает зону и разрешенные сервисы

Дополнительно: `sudo nft list ruleset` - показывает правила nftables напрямую

На базовом уровне важно понять, открыт ли нужный порт для нужного направления

Не стоит отключать firewall полностью, если можно

Проверка с клиента должна повторять исходный сценарий

`curl http://127.0.0.1/` проверяет приложение только с самого сервера

`curl http://192.168.10.20/` с другой VM проверяет доступность по сети

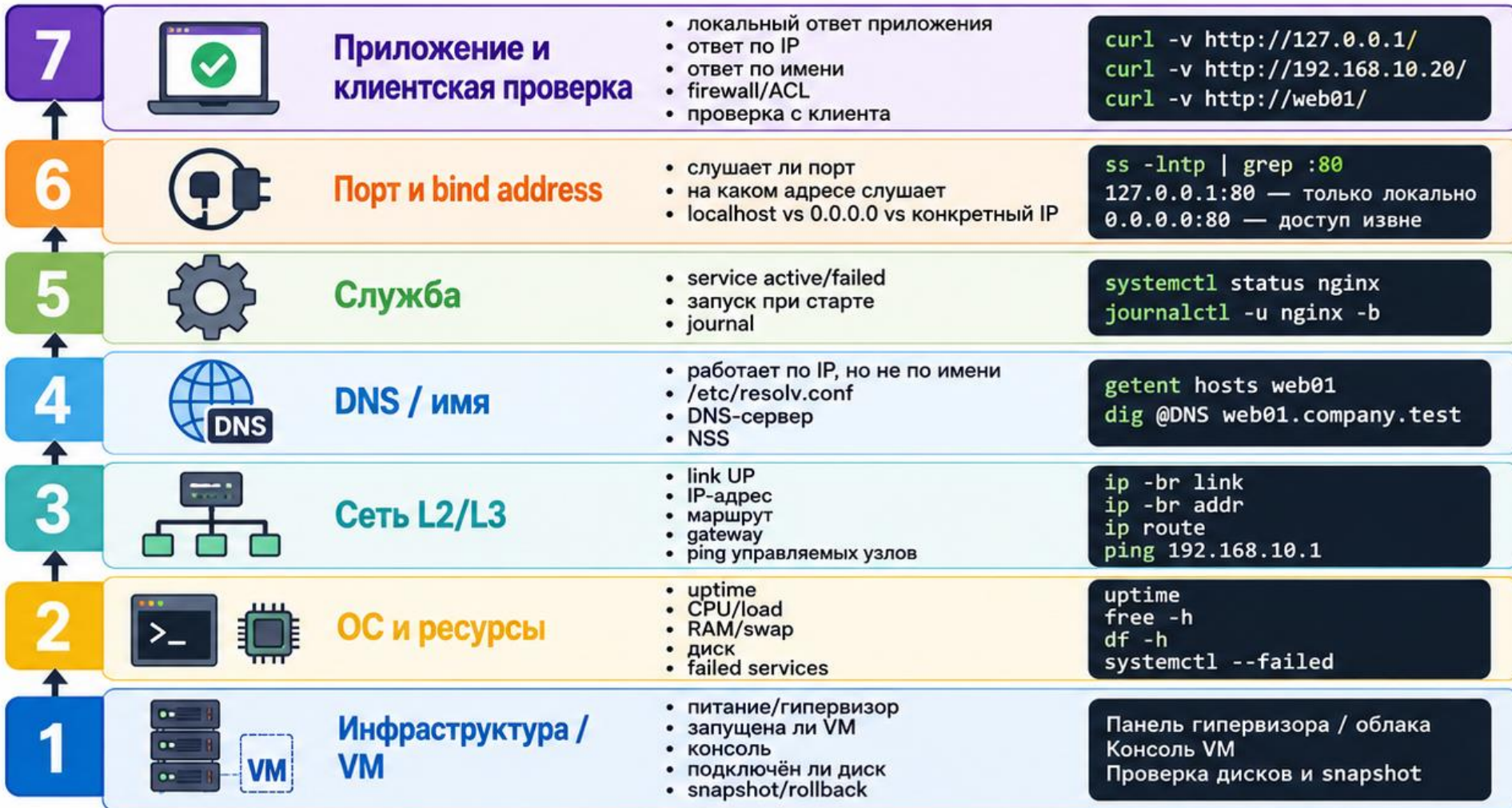
Если локально работает, а с клиента нет, проверяют `bind`, `firewall`, `route` и `ACL`

Если с клиента работает по IP, но не по имени, проверяют `DNS`

Исправление доказано только тем тестом, который раньше воспроизводил проблему

Диаграмма: диагностика сервера по уровням

Проверка идёт по уровням: от инфраструктуры и ОС к сети, службе, порту, приложению и клиенту.



Как читать уровни



Если нижний уровень неисправен — выше проверять рано



Каждый шаг сужает область поиска



Событие рядом по времени — не доказательство причины



Исправление подтверждаем повторением сценария клиента

Логика диагностики



Причина подтверждена, когда найдено отклонение и проверка после исправления успешна.

7. Перед изменением нужна точка возврата

Rollback - возврат к предыдущему рабочему состоянию после неудачного изменения

Backup → Edit → Syntax Check → Apply → Test →
Rollback if needed

Backup должен быть готов до изменения, а не после появления новой ошибки

Для конфигурационных файлов важно сохранять права и атрибуты

Если меняется несколько файлов, копируют все изменяемые файлы или каталог конфигурации

.bak, snapshot, backup и Git решают разные задачи

.bak - быстрый локальный откат одного файла
конфигурации

VM Snapshot - удобная учебная точка возврата для
Break/Fix-лаборатории

Backup - отдельная восстанавливаемая копия данных
или конфигурации

Git полезен для истории текстовых конфигураций, но
не заменяет backup

На базовом уровне студент должен понимать
назначение каждого инструмента, а не смешивать их

Безопасное изменение выполняется по короткому сценарию

`sudo cp -a app.conf app.conf.bak` - сохраняет копию с атрибутами

`sudoedit app.conf` - безопасно открывает файл на редактирование от имени администратора

`sudo nginx -t` - проверяет синтаксис перед применением

`sudo systemctl reload nginx` - применяет рабочую конфигурацию без полного restart

`curl http://127.0.0.1/` - проверяет локальный HTTP-ответ после изменения

Если тест неуспешен, возвращают `app.conf.bak` и

8. Отчет должен доказывать причину

Формат: Симптом → Гипотеза → Проверка →

Фактический вывод

Продолжение: Причина → Исправление → Проверка
результата → Rollback

В отчет не нужно переносить весь терминал

Достаточно ключевых строк вывода, которые
подтверждают вывод администратора

Отдельно фиксируют, где находится backup и как
выполнить rollback

Хороший отчет позволяет другому администратору
повторить логику проверки

Пример web01: от симптома к причине

Симптом: web01 недоступен по HTTP с client01
`systemctl status nginx` - показывает, что служба Nginx не запущена или failed

`sudo ss -ltnp | grep :80` - проверяет, слушается ли TCP 80

`sudo nginx -t` - проверяет конфигурацию и показывает строку с ошибкой

Фактический вывод: причина связана не с сетью, а с ошибкой конфигурации Nginx

Перед исправлением создают backup изменяемого файла

Пример web01: исправление и доказательство результата

Исправление: вернуть корректную директиву или закрывающую скобку в app.conf

`sudo nginx -t` - повторно подтвердить корректный синтаксис

`sudo systemctl reload nginx` - применить исправленную конфигурацию

`curl http://127.0.0.1/` - проверить локальный ответ приложения

`curl http://192.168.10.20/` или `curl`

`http://web01.company.test/` с клиента - повторить исходный сценарий

Break/Fix-лабораторная строится как доказательство

Web: syntax error в Nginx или bind только на 127.0.0.1

Disk: заполнен /var или закончились inode

Network: отсутствует default route или указан неправильный DNS

Resources: CPU pressure или RAM pressure

Для каждого сценария: симптом, гипотеза, команда проверки, причина, backup, исправление, проверка
Задача студента - не просто починить, а доказать, почему именно это было причиной

Эскалация - это профессиональная часть диагностики

Нужно остановиться, если есть риск потери данных

Нужно эскалировать, если проблема затрагивает несколько систем

Нельзя запускать непонятную команду от root на рабочем сервере

Если нет прав на нужный компонент, это фиксируют в отчете

Если проблема выходит за пределы компетенции, администратор передает ее старшему специалисту с фактами проверки

Итоговый алгоритм диагностики Linux-сервера

1. Уточнить симптом → 2. Проверить доступность → 3.

Проверить ресурсы

4. Проверить service → 5. Проверить port → 6.

Проверить logs

7. Проверить network, DNS и firewall

8. Сделать backup → 9. Исправить → 10. Повторить
исходный тест

Каждый шаг должен давать факт, который сужает
область поиска

Финальный результат - подтвержденная причина и
повторяемое доказательство исправления

Контрольные вопросы

1. Чем симптом отличается от причины?
2. Что такое baseline?
3. Почему высокий load не всегда означает перегрузку CPU?
4. Как проверить RAM и swap?
5. Как последовательно проверить service, port и application?
6. Что проверять, если по IP работает, а по имени нет?
7. Что проверять, если локально работает, а с клиента нет?
8. Зачем делать syntax check?